

Uppgift 1

I den första uppgiften skapade jag klasserna Vertex, Edge och Graph för att representera en graf i Java. Jag använde det givna interfacet som mall för vilka metoder som behövde finnas i Graph-klassen. Jag använde också GraphViewer för att kontrollera att grafen gick att rita upp och att noder, kanter och avstånd visades korrekt.

I min implementation används Vertex för att representera en nod med information, koordinater och färg. Edge representerar en riktad kant mellan två vertices och beräknar avståndet mellan dem med hjälp av Pythagoras sats. Graph använder två HashMap-strukturer: en för att lagra vertices och en för att lagra kanter kopplade till varje vertex. Eftersom grafen är oriktad skapas två riktade Edge-objekt när en kant läggs till.

Jag använde AI som stöd under arbetet, främst för att förstå hur HashMap kunde användas för att koppla en vertex till dess kanter. Jag behövde också hjälp med att tänka kring remove-metoden, eftersom den inte bara ska ta bort en vertex utan även alla kanter som är kopplade till den. Jag testade sedan implementationen med JUnit och körde även GraphViewer för att se grafen visuellt.

Uppgift 2

Dijkstras algoritm används för att hitta den kortaste vägen från en startnod till andra noder i en viktad graf. En viktad graf innebär att varje kant har ett värde, till exempel avstånd, tid eller kostnad. I min graf kan värdet distance i Edge-klassen användas som vikt.

Algoritmen börjar med att startnoden får avståndet 0 och alla andra noder får ett väldigt stort avstånd. Sedan väljs den obesökta nod som har kortast känt avstånd. Från den noden undersöks alla grannar. Om en kortare väg hittas till en granne uppdateras avståndet, och man sparar också vilken nod man kom ifrån. Detta fortsätter tills målnoden har hittats eller tills alla relevanta noder är behandlade.

För att använda Dijkstra i min grafimplementation skulle jag kunna lägga till en metod i Graph, till exempel shortestPath(T start, T goal). Under algoritmen behöver man hålla reda på kortaste kända avstånd, tidigare vertex och vilka vertices som redan har behandlats. Jag skulle helst spara detta i lokala HashMap-strukturer i metoden, i stället för som permanenta attribut i Vertex, eftersom grafen då kan användas flera gånger med olika startnoder.

Jag skulle inte behöva lägga till så mycket i Edge, eftersom den redan har distance, vilket kan användas som kantens vikt. Man skulle däremot kunna använda en PriorityQueue i Graph för att enklare välja nästa vertex med lägst avstånd.

Typen T kan till exempel vara ett Ortsnamn, ett id-nummer eller någon annan unik identifierare. I min implementation används T info som nyckel i HashMap, och därför behövs den för att hitta rätt vertex.

För att visualisera resultatet skulle man kunna använda setColor() i Vertex och Edge. Till exempel kan kanterna i den kortaste vägen färgas röda, medan besökta noder kan få en annan färg. Eftersom GraphViewer redan använder färgattributen skulle resultatet kunna visas direkt i GUI:t.